

Capítulo 1

PWN College - Computer 101

📌 Estructura y Contenido del Capítulo

1.1. Your First Program	1
1.2. Computer Memory	5
1.3. Software Introspection	6
1.4. Output And Input	8
1.5. Control Flow	15
1.6. Endian Escapades	25

1.1. Your First Program

☰ Subsecciones del Tema

- Video - Computer Architecture
- Video - Assembly
- Video - Registers
- Your First Register
- Your First Syscall
- Exit Codes
- Building Executable
- Moving Between Registers

1.1.1. Video - Computer Architecture

Ok cuando es que se ejecuta un programa, este para que se le pueda pasar a la computadora, tiene que ser transcrito a lenguaje máquina (binario) y pues bueno dependiendo de cual es que sea el lenguaje de programación, este va a ser interpretado o compilado.

Ahora dentro de un CPU, vamos a contar con las compuertas lógicas, siendo las más comunes la compuerta AND, OR y NOT.

1.1.2. Video - Assembly

Speaking Binary, para nosotros los humanos es complejo el poder hablar binary code, por eso se creó una representación de texto de dicho binary code, al cual lo llamamos Assembly.

Luego el video también habla sobre esto de aquí:

1.1.3. Video - Registers

La CPU necesita ser veloz, para esto necesita poder acceder rápidamente a la información almacenada.

All roads lead to the CPU

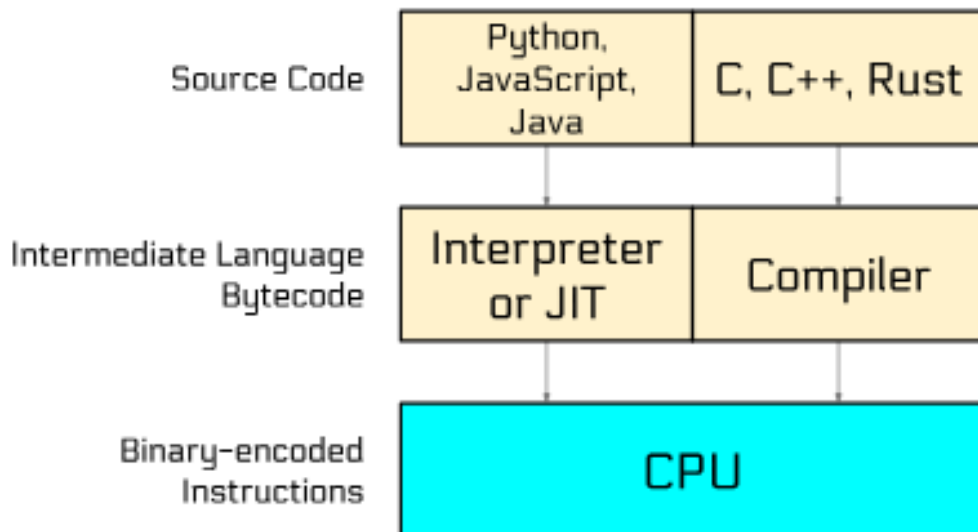


Figura 1.1: All roads to the CPU

Digging Deeper

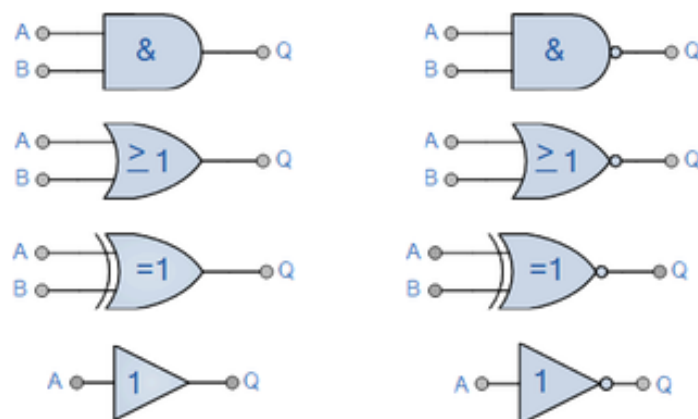


Figura 1.2: Digging Deeper

A esto es lo que se le conoce como registros, y contamos con múltiples de ellos siendo llamados de propósito general, el cual depende de la arquitectura de la CPU.

- Su tamaño suele ser de 64 bits para los computadores modernos.
- Podemos acceder de forma parcial a los registros, los primeros 32, 16, 8.

Bueno también hace mención que existen otros tipos de registros más complejos.^o difíciles de trabajar, pero se verá más adelante (creo xD).

Nouns / Operands

What types of nouns might we deal with? Data!

For the most part, the CPU is concerned with three types of data:



data we directly give it as part of the instruction



data that is close at hand



data in storage

Figura 1.3: Nouns / Operands

All partial accesses on amd64 (that I know of)

64	32	16	BH	BL
rax	eax	ax	ah	al
rcx	ecx	cx	ch	cl
rdx	edx	dx	dh	dl
rbx	ebx	bx	bh	bl
rsp	esp	sp		spl
rbp	ebp	bp		bpl
rsi	esi	si		sil
rdi	edi	di		dil
r8	r8d	r8w		r8b
r9	r9d	r9w		r9b
r10	r10d	r10w		r10b
r11	r11d	r11w		r11b
r12	r12d	r12w		r12b
r13	r13d	r13w		r13b
r14	r14d	r14w		r14b
r15	r15d	r15w		r15b

Figura 1.4: All partial accesses on amd64

1.1.4. Your First Register

Los registros contienen la data. La CPU puede poner data en los registros, moverla entre registros y demás. Así mismo, para el Assembly que se estará trabajando es el x86_64, donde contamos con 16 registros de propósito general, como `rax`.

Almacenar un valor a un registro:

```
</>
```

```
mov rax, 60
```

Register Arithmetic

Once you have data in registers, you can *compute*!

For most arithmetic instructions, the first specified register stores the result.

Instruction	C / Math equivalent	Description
add rax, rbx	$rax = rax + rbx$	add rax to rbx
sub ebx, ecx	$ebx = ebx - ecx$	subtract ecx from ebx
imul rsi, rdi	$rsi = rsi * rdi$	multiple rsi to rdi, truncate to 64-bits
inc rdx	$rdx = rdx + 1$	increment rdx
dec rdx	$rdx = rdx - 1$	decrement rdx
neg rax	$rax = 0 - rax$	negate rax in terms of numerical value
not rax	$rax = \sim rax$	negate each bit of rax
and rax, rbx	$rax = rax \& rbx$	bitwise AND between the bits of rax and rbx
or rax, rbx	$rax = rax rbx$	bitwise OR between the bits of rax and rbx
xor rcx, rdx	$rcx = rcx \wedge rdx$	bitwise XOR (don't confuse $\wedge$ for exponent!)
shl rax, 10	$rax = rax \ll 10$	shift rax's bits left by 10, filling with 10 zeroes on the right
shr rax, 10	$rax = rax \gg 10$	shift rax's bits right by 10, filling with 10 zeroes on the left
sar rax, 10	$rax = rax \gg 10$	shift rax's bits right by 10, with sign-extension to fill the now 'missing' bits!
ror rax, 10	$rax = (rax \gg 10) (rax \ll 54)$	rotate the bits of rax right by 10
rol rax, 10	$rax = (rax \ll 10) (rax \gg 54)$	rotate the bits of rax left by 10

Figura 1.5: Registers Arithmetic

1.1.5. Your First Syscall

Los programas interactúan con la CPU usando instrucciones de assembly, pero para que puedan interactuar con el sistema operativo usan los `syscall` o *System Call Instruction*.

En Linux contamos alrededor de 330 diferentes `syscall`.

1.1.6. Exit Codes

Los programas, cuando finalizan, pueden salir con un código de salida al finalizar. Este código pasa por el registro `rdx`, el cual indica cuál es dicho código.



```
mov rdx, 42
mov rax, 60
syscall
```

1.1.7. Building Executable

Para compilar un archivo asm (`.s` o `.S`), vamos a usar el comando `as`. Y para convertirlo en un ejecutable se tiene que usar el comando `ld`.

La directiva `.intel_syntax noprefix`, es usada para indicar que el estilo de escritura es el de Intel y no queremos usar prefijos especiales para cada instrucción.



```
.intel_syntax noprefix
mov rdi, 42
mov rax, 60
syscall
```

1.1.8. Moving Between Registers



```
.intel_syntax noprefix
mov rdi, rsi
mov rax, 60
syscall
```

1.2. Computer Memory

Subsecciones del Tema

- › Video - (review) Computer Architecture
- › Video - Memory
- › Loading From Memory
- › More Loading Practice
- › Dereferencing Pointers
- › Dereferencing Yourself
- › Dereferencing with Offsets
- › Stored Addresses
- › Double Dereference
- › Dictionary

1.2.1. Video - (review) Computer Architecture

Una recapitulación del capítulo anterior.

1.2.2. Video - Memory

1.2.3. Loading From Memory

La memoria al ser un lugar muy grande, esta se almacena de forma secuencial, por lo tanto cuando realizamos lo siguiente:



```
mov rdi, 31337
```

En este caso, almacenamos el valor de 31337 en el registro de `rdi`.
Pero si queremos saber el valor del registro `31337`:



```
mov rdi, [31337]
```

Entonces, aquí estamos moviendo el valor del registro `31337` al valor de `rdi`.

1.2.4. More Loading Practice

Más práctica, aquí.

1.2.5. Dereferencing Pointers

1.2.6. Dereferencing Yourself

Podemos realizar la desreferencia, de un registro en el mismo registro.



```
mov rdi, [rdi]
```

1.2.7. Dereferencing with Offsets

El valor de `[rdi]`, es el valor de un registro de la memoria. Por lo tanto, se puede realizar un aumento, para poder buscar un valor adicional.



```
mov rdi, [rdi + 8]
```

1.2.8. Stored Addresses

1.2.9. Double Dereference

1.2.10. Dictionary

English	Spanish
word	palabra
word	palabra

1.3. Software Introspection

Subsecciones del Tema

- › Disassembling Programs
- › Tracing Syscalls
- › Starting GDB
- › Quitting GDB
- › Starting Programs in GDB
- › Disassembling in GDB
- › Stepping Through Instructions
- › Reading Register Values
- › Setting Register Values
- › Popping Stack Values
- › Examining Memory
- › Examining Stack Pointers
- › Cooperate Debugging
- › Running with Arguments
- › Redirecting Input in GDB

1.3.1. Disassembling Programs

Vamos a usar unas de las herramientas más comunes, que es `objdump`, y se usará las siguientes banderas de ejecución:

- `-d`: Para que haga un disassembling, de las secciones que tiene instrucciones de ensamblador.
- `-M intel`: Es para escoger la sintaxis a la cual le vamos a realizar disassembling.



```
objdump -d -M intel /tmp/your_program
```

Y vamos a tener una salida similar a esta de aquí:



```
hacker@dojo:~$ objdump -d -M intel /tmp/your-program

/tmp/your-program:      file format elf64-x86-64

Disassembly of section .text:

0000000000401000 <_start>:
401000: 48 c7 c7 39 05 00 00  mov    rdi,0x539
401007: 48 c7 c7 00 00 00 00  mov    rdi,0
40100e: 48 c7 c0 3c 00 00 00  mov    rax,0x3c
401015: 0f 05                  syscall
```

```
ubuntu@introspecting-disassembling-programs:~$ objdump -d -M intel /challenge/disassemble-me
/challenge/disassemble-me:      file format elf64-x86-64

Disassembly of section .text:

0000000000401000 <_bss_start-0x1000>:
401000: 48 c7 c7 4d 6d 00 00  mov    rdi,0x6d4d
401007: 48 c7 c7 00 00 00 00  mov    rdi,0x0
40100e: 48 c7 c0 3c 00 00 00  mov    rax,0x3c
401015: 0f 05                  syscall
ubuntu@introspecting-disassembling-programs:~$ /challenge/submit-number
You must run /challenge/submit-number with the secret value you retrieved from
the challenge.

Usage: /challenge/submit-number [SECRET_VALUE]
ubuntu@introspecting-disassembling-programs:~$ /challenge/submit-number 0x6d4d
CORRECT! Here is your flag:
pwn.college{0ftSiD9RpGCaCG41rswH1LN0C95.0LMzczMywSMycjN4EzW}
ubuntu@introspecting-disassembling-programs:~$
```

Figura 1.6: Disassembling Programs Result

1.3.2. Tracing Syscalls

Con el programa `strace` nos permite poder ver las llamadas a `syscall`, que realiza un programa.



```
hacker@dojo:~$ strace /tmp/your-program
execve("/tmp/your-program", ["/tmp/your-program"], 0x7ffd48ae28b0 /* 53 vars */) = 0
exit(42)                                = ?
+++ exited with 42 +++
hacker@dojo:~$
```

En este caso, vemos que al momento de ejecutar `strace`, nos muestra la ejecución de los `syscall`, en un formato parecido a las funciones en C.



```
system_call(parameter, parameter, parameter, ... )
```

Donde los `parameter` son los valores que se le manda a ejecutar.

1.3.3. Starting GDB

GDB = GNU Debugger

Ejecución del GDB:



```
gdb /path/to/file
```

1.3.4. Quitting GDB

Para salir del GDB, tenemos que escribir quit, o q.

1.3.5. Starting Programs in GDB

Con el comando `starti` en GDB, se indica que el programa empiece a ejecutarse, desde la línea de código, y vamos a poder ver su comportamiento, en tiempo de ejecución.

Por ejemplo, vamos a poder desensamblar el programa con `disassemble`.

1.3.6. Disassembling in GDB

Lo mismo que el código pasado, no entender el porqué, pero bueno.

1.3.7. Stepping Through Instructions

Podemos realizar la ejecución paso a paso, con `stepi`, o usando la versión corta con `si`.

1.3.8. Reading Register Values

1.3.9. Setting Register Values

1.3.10. Popping Stack Values

1.3.11. Examining Memory

1.3.12. Examining Stack Pointers

1.3.13. Cooperate Debugging

1.3.14. Running with Arguments

1.3.15. Redirecting Input in GDB

1.4. Output And Input

Subsecciones del Tema

- › Video - System Calls
- › Writing Output
- › Chaining Syscalls
- › Writing String
- › Reading Data

- › Reading Exactly
- › Opening Files
- › Hardcoding the Filename
- › RIP - Relative Strings

1.4.1. Video - System Calls

1.4.2. Writing Output

Recordando los *File Descriptors*(*FDs*), vemos que cada proceso empieza con alguno de estos 3 *FDs*:

- **FD 0:** Standard Input
- **FD 1:** Standard Output
- **FD 2:** Standard Error

En el caso de la llamada al sistema para **exit**, tenemos que especificar que el valor de `mov rdi`, 42. Ahora para poder ejecutar la llamada al sistema de **write**, tenemos que establecer el valor de `rdi` en 1 o en 2.

Ahora, este no es el único parámetro que requiere la instrucción **write**, esta también necesita saber en que parte de la memoria esta el texto y cuantos caracteres vamos a leer. Similar a como sería una función en C:

</> Ejemplo de la función write

```
write(file_descriptor, memory address, number_of_characters_to_write)
# Ejemplo de su uso
write(1, memory address,10)
```

Ahora, por estándar, esto de aquí ya esta definido:

- El primer parámetro siempre será **rdi**.
- El segundo parámetro siempre será **rsi**.
- El tercer parámetro siempre será **rdx**.

</>

```
mov rax , 1
mov rdi , 1
mov rsi , [rsp + 16]
mov rdx , 1
syscall
```

🚩 Flag Capturada

✅ CAPTURADA

```
pwn.college{MkMD83Ee-aMqL23-gYH-nLVAcvF.QXwUTN2wSMycjN4EzW}
```

1.4.3. Chaining Syscalls

En el código anterior, si lo compilamos y ejecutamos, vamos a ver que el programa se detiene, para que se no bloquee, se tiene que agregar la llamada de salida.



```
mov rax , 1
mov rdi , 1
mov rsi , [rsp + 16]
mov rdx , 1
syscall
mov rax , 60
mov rdi , 42
syscall
```

Flag Capturada

✓ CAPTURADA

pwn.college{k3RvbIqm0S-vpMk7rE2EgfBpya-.QXxUTN2wSMycjN4EzW}

1.4.4. Writing String

Práctica para poder leer, más de un solo caracter, ya que en el ejemplo del capítulo 1.4.2, es para un solo digito, en este caso solo se tendría que cambiar el valor de **rdx**



```
mov rax , 1
mov rdi , 1
mov rsi , [rsp + 16]
mov rdx , 64
syscall
mov rax , 60
mov rdi , 42
syscall
```

Flag Capturada

✓ CAPTURADA

pwn.college{g79tV7cz1AjIFS6t816QBs5NXqM.01NzEDMxwSMycjN4EzW}

1.4.5. Reading Data

Ahora para poder leer, vamos a usar de ejemplo como es en el lenguaje C.

Reading syscall

```
read(0, some_address, 5)
```

En este caso, como vemos a partir de *some_address* se va a empezar a a rellenar, hasta leer los primeros 5 caracteres del texto.

Address	Contents
some_address	48
some_address+1	45
some_address+2	4c
some_address+3	4c

```
| some_address+4 | 4f |  
+-----+-----+
```

Aquí un ejemplo de su como sería leer, e imprimir un texto de 128 caracteres:

</>

```
mov rax , 0  
mov rdi , 0  
mov rsi , rsp  
mov rdx , 128  
syscall
```

```
mov rax , 1  
mov rdi , 1  
mov rsi , rsp  
mov rdx , 128  
syscall  
mov rax , 60  
mov rdi , 42  
syscall
```

Flag Capturada

✓ CAPTURADA

pwn.college{8hX8fIcaF2n1GfRHFQCGNA0JmK1.0FNwUTNxwSMycjN4EzW}

1.4.6. Reading Exactly

Ahora, anteriormente como hemos visto anteriormente, hemos leído y escrito un mensaje de 128 caracteres, pero no siempre vamos a saber cuantos caracteres puede tener el texto que vamos a leer.

La funcion **read**, nos regresa el valor de la cantidad de caracteres que realmente a leído, y este se almacena en **rax**.

</>

```
mov rax , 0  
mov rdi , 0  
mov rsi , rsp  
mov rdx , 128  
syscall
```

```
mov rdx, rax
```

```
mov rax , 1  
mov rdi , 1  
mov rsi , rsp  
syscall  
mov rax , 60  
mov rdi , 42  
syscall
```

Flag Capturada

✓ CAPTURADA

pwn.college{8tnm880ONdKT9_M4-W93McmxZWn.0VM4ATNywSMycjN4EzW}

1.4.7. Opening Files

Para abrir un archivo, que se encuentre en algun lugar de nuestro disco, vamos a necesitar acceso a dicho archivo, para esto necesitamos abrirlo.

</> Open file

```
open("/flag",0) # 0 default read-only
```

Ahora similar a la función **read**, la función **open** nos esta devolviendo un valor en **rax**, el cual contiene el **fd** del archivo que acabamos de leer, y se tiene que usar este de aquí para poder leer el archivo.

</> m

```
ov rax, 2
mov rdi, [rsp+16]
mov rsi, 0
syscall

mov rdi, rax
mov rax, 0
mov rsi, rsp
mov rdx, 256
syscall

mov rdx, rax
mov rax, 1
mov rdi, 1
mov rsi, rsp
syscall

mov rax, 60
mov rdi, 42
syscall
```

Nota Informativa

En este caso, se esta pasando la dirección del archivo, como parametro, por eso es que se lee desde **rsp + 16**.

Flag Capturada

 CAPTURADA

```
pwn.college{UYeD10tcieFHPjdDycMVcs0l_aj.0VM0czMywSMycjN4EzW}
```

1.4.8. Hardcoding the Filename

En el caso anterior 1.4.7, pasamos el valor del archivo como parametros, pero si queremos pasar el nombre de un archivo fijo, sería de la siguiente manera.



```
intel_syntax noprefix
```

```
mov BYTE PTR [rsp], '/'  
mov BYTE PTR [rsp+1], 'f'  
mov BYTE PTR [rsp+2], 'l'  
mov BYTE PTR [rsp+3], 'a'  
mov BYTE PTR [rsp+4], 'g'  
mov BYTE PTR [rsp+5], 0
```

```
mov rax, 2  
mov rdi, rsp  
mov rsi, 0  
syscall
```

```
mov rdi, rax  
mov rax, 0  
mov rsi, rsp  
mov rdx, 128  
syscall
```

```
mov rdx, rax  
mov rax, 1  
mov rdi, 1  
mov rsi, rsp  
syscall
```

```
mov rax, 60  
mov rdi, 42  
syscall
```

🚩 Flag Capturada

✅ CAPTURADA

```
pwn.college{ksqeBnvOJWvAvIls3V_A_gdsD9k.0lM0czMywSMycjN4EzW}
```

1.4.9. RIP - Relative Strings

El problema que presentamos en la sección anterior 1.4.8, es que si escribimos un error al escribir la dirección del texto, sería muy teidoso encontrar su error, o si el nombre del archivo es muy largo.

Por lo que, en el código ensamblador, se puede contener bytes que no esten destinados a ejecutarse, estos serían colocados después de la llamada al sistema **exit**.

🔗 Ejemplo de bits adicionales

```
_start:  
...  
mov rax, 60  
syscall  
path:  
.asciz "/flag"
```

En este caso, la directiva **.asciz** va a contener los bytes de la cadena, junto con el 0 de la terminación.

Y la etiqueta **path** marca donde comienzan esos bytes.

Esto era factible en programas anteriores, donde los programas siempre se cargaban en la misma dirección de memoria, por lo que se podría escribir esto de aquí:

</>

```
_start:
...
mov rdi, path
path:
.asciz "/flag"
```

Esto de aquí no funciona, porque ahora los programas con compilados por un tema de seguridad, lo que impide conocer en que parte de la memoria es que se encuentra almacenado.

Para esto de aquí, se creo el *Instruction Pointer Relative Addressing*(Dirección Relativo al Puntero de Instrucción) El puntero de instrucción **rip**, va a contener la dirección de la siguiente instrucción que se ejecutará en la CPU.

</>

```
_start:
...
mov rdi, [rip;path]
...
path:
.asciz "/flag"
```

Esto último presenta el problema que el valor de **rdi**, va a ser **/flag**", no la dirección de memoria donde este esta almacenado, para esto se tiene que utilizar el la instrucción **lea**.

La instrucción **lea**(*Load Effective Address*), permitiendo que se haga referencia a que la CPU calcula todas las operaciones necesarias, como sumar un desplazamiento al puntero de instrucción.

</>

```
_start:
...
lea rdi, [rip;path]
...
path:
.asciz "/flag"
```

</> Solución del problema

```
.intel_syntax noprefix

mov rax, 2
lea rdi, [rip + path]
mov rsi, 0
syscall

mov rdi, rax
mov rax, 0
mov rsi, rsp
mov rdx, 128
syscall

mov rdx, rax
mov rax, 1
mov rdi, 1
mov rsi, rsp
syscall

mov rax, 60
mov rdi, 42
syscall

path:
.asciz "/flag"
```

🚩 Flag Capturada**✓ CAPTURADA**

pwn.college{c45qx3vAQJ53UI51AOM6qFuPtYE.01N3ATNywSMycjN4EzW}

1.5. Control Flow

☰ Subsecciones del Tema

- › Video - Registers
- › Video - Control Flow
- › Comparing Values
- › Comparing Characters
- › Conditional Control Flow
- › Comparing Strings
- › Reverse the Password
- › Conditionals Without Conditionals
- › Recognizing Loops
- › Writing Loops
- › Writing From a Shared Library
- › Returning a Value
- › Calling Through a Pointer
- › Calling Through a Pointer with an Argument
- › Saving Caller - Saved Registers
- › Saving Callee - Saved Registers

1.5.1. Video - Registers

Este video ya se abordó anteriormente en [1.1.3](#).

1.5.2. Video - Control Flow

1.5.3. Comparing Values

Para poder realizar las comparaciones, se utiliza la instrucción **cmp**, este compara 2 valores restando el segundo operando del primero. El valor de dicha resta, no se almacena en ningún

lugar al que se pueda acceder directamente, este actualiza las **flag** internas de la CPU.

</> Ejemplo

```
cmp rdi, 42 ; Se comprar un valor inmediato
cmp QWORD PTR [rsp], 42 ; tambien se puede comparar direcciones de memoria
```

Esto de aquí, calcula internamente $rdi - 42$ y la CPU establece un bit especial llamado **Zero Flag (ZF)**, en caso de que la resta sea 0 el valor de **ZF** es 1, caso contrario va a tomar el valor de 0.

Con lo anteriormente mencionado x86 nos proporciona una familia de instrucciones que escriben 0 o 1 en un destino de 1 byte.

</>

```
setz dil
```

Esto comprueba el valor de **ZF**, y lo escribe en **dil**. Y a su vez, **setnz(Set if Not Zero)** que hace lo contrario.

Recordar que **dil** son los primeros 8 bits menos significativos del registro **rdi**.

</> Solución del Problema

```
.intel_syntax noprefix
cmp QWORD PTR [rsp], 42
setz dil
mov rax, 60
syscall
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{s9M8ey0NvS8aym8WgNaOwwTzosR.01M0czMywSMycjN4EzW}
```

1.5.4. Comparing Characters

En el módulo de Stack, se vio que `[rsp + 16]` es un puntero al primero valor pasado como parámetro **argv[1]**. Y como vimos en el capítulo anterior 1.4 podemos solo utilizar el valor de un solo byte (**BYTE PTR [rax]**), y con esto podemos realizar comparaciones de un solo caracter.

</> Solución

```
.intel_syntax noprefix
mov rax, [rsp + 16]
cmp BYTE PTR [rax], 'p'
setz dil
mov rax, 60
syscall
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{EK0Xt5rxYiy5yQWdKCGSVY3dSbL.0FN0czMywSMycjN4EzW}
```

1.5.5. Conditional Control Flow

Con el resultado anterior, se le puede indicar a la CPU que salte a otra parte del código.

Estos se ejecutan después de la instrucción `cmp`, si quiere revisar todos los [saltos condicionales](#).

Aquí una tabla resumen:

Instrucción	Mnemónico / Significado	Condición lógica	Banderas evaluadas
<code>jmp</code>	Jump	Incondicional	Ninguna (salta siempre)
<code>je / jz</code>	Jump if Equal / Zero	$A == B$ o resultado cero	$ZF = 1$
<code>jne / jnz</code>	Jump if Not Equal / Not Zero	$A \neq B$ o resultado no cero	$ZF = 0$
<code>jg / jnle</code>	Jump if Greater	$A > B$ (con signo)	$ZF = 0$ y $SF = 0F$
<code>jge / jnl</code>	Jump if Greater or Equal	$A \geq B$ (con signo)	$SF = 0F$
<code>jl / jnge</code>	Jump if Less	$A < B$ (con signo)	$SF \neq 0F$
<code>jle / jng</code>	Jump if Less or Equal	$A \leq B$ (con signo)	$ZF = 1$ o $SF \neq 0F$
<code>ja / jnbe</code>	Jump if Above	$A > B$ (sin signo)	$CF = 0$ y $ZF = 0$
<code>jae / jnb</code>	Jump if Above or Equal	$A \geq B$ (sin signo)	$CF = 0$
<code>jb / jc</code>	Jump if Below / Carry	$A < B$ (sin signo) / Acarreo	$CF = 1$
<code>jbe / jna</code>	Jump if Below or Equal	$A \leq B$ (sin signo)	$CF = 1$ o $ZF = 1$
<code>js</code>	Jump if Sign	Resultado negativo	$SF = 1$
<code>jns</code>	Jump if Not Sign	Resultado positivo o cero	$SF = 0$
<code>jo</code>	Jump if Overflow	Desbordamiento con signo	$OF = 1$
<code>jno</code>	Jump if Not Overflow	Sin desbord. con signo	$OF = 0$

Tabla 1.1: Algunos saltos en ensamblador x86-64

```
</>
```

```
cmp BYTE PTR [rax], [p]
jne fail
```

fail es una **label**, estas no son una instrucción de máquina, sino como indicador para que el CPU sepa a que parte tiene que saltar para continuar con la ejecución.

```
</>
```

```
fail:
mov rdi, 1
mov rax, 60
syscall
```

En caso de que no se origine el salto, va a continuar con la ejecución normal.

</> Ejemplo de uso

```
main:
[load and compare]
jne fail      ;jump to fail if NOT equal

success:
mov rdi, 0
mov rax, 60
syscall

fail:
mov rdi, 1
mov rax, 60
syscall
```

</> Solución del problema

```
.intel_syntax noprefix
mov rax, [rsp + 16]
cmp BYTE PTR [rax], 'p'
jne fail

mov rdi, 0
mov rax, 60
syscall

fail:
mov rdi, 1
mov rax, 60
syscall
```

🚩 Flag Capturada**✓ CAPTURADA**

pwn.college{QZ9khdqQojrEPg_83_78vJ54Ecc.0VN0czMywSMycjN4EzW}

1.5.6. Comparing Strings

Para poder comparar string, se tiene que comparar letra por letra:

</>

```
mov rax, [rsp + 16]

cmp BYTE PTR [rax], 'V'
jne fail

cmp BYTE PTR [rax+1], 'E'
jne fail

cmp BYTE PTR [rax+2], 'S'
jne fail
```

Flag Capturada

✓ CAPTURADA

pwn.college{w_w8UjT_9EaU1i1dN9bT0-Y5gF9.01N0czMywSMycjN4EzW}

1.5.7. Reverse the Password

Este desafío busca recordar los temas anteriores, como el objdump, y lo que hemos visto anteriormente para la comparación de caracteres y el poder leer código en **ASM**.

i Consideraciones

Al momento de realizar comparaciones, también se puede hacer con el valor en hexadecimal su valor en ASCII a comparar

</> Comparacion con valor en hexadecimal

```
cmp BYTE PTR [rax], 0x70 ; 0x70 = 112 = 'p'
```

Flag Capturada

✓ CAPTURADA

pwn.college{wNjWDbSIItWSvndFqwZshHkx80-.01N0czMywSMycjN4EzW}

1.5.8. Conditionals Without Conditionals

Para poder realizar una serie de condicionales, se suele trabajar con una **jump table** en vez de tener una secuencia de condicionales, una detrás de otra.

Si desamblamos un programa vamos a ver algo así:

</>

```
mov    rax, 0                ; zero out rax
mov    al, BYTE PTR [rcx]    ; load the character into the low byte of rax
mov    rax, [rax*8+0x1234000] ; load a stored address from the jump table at 0x1234000
jmp    rax                   ; jump to it
```

En este caso, el valor de **0x1234000**, representa la dirección donde se encuentra la tabla de saltos.

En este problema, nos piden:

1. Encontrar la dirección de la tabla de saltos, con objdump
2. Con el GDB detectar cual es la dirección correcta a la que tiene que saltar para que salga exitosamente.
3. calcular el valor correcto, con la siguiente formula Índice = $\frac{\text{Dirección Slot} - \text{Dirección Base}}{8}$

Flag Capturada

✓ CAPTURADA

pwn.college{QuJcHz6h6ZmYjOh93E_-XKn_J3b.0F00czMywSMycjN4EzW}

1.5.9. Recognizing Loops

Un bucle(*loop*) es una secuencia de instrucciones que vuelve atrás para repetirse.

</> Ejemplo de un bucle

```
loop:
mov    al, BYTE PTR [rsi]      ; load next password character
cmp    al, BYTE PTR [rdi]      ; compare against next argv[1] character
jne    fail                    ; mismatch → jump to fail
cmp    al, 0x0                 ; reached the null terminator?
je     success                 ; yes → all characters matched!
inc    rdi                     ; **inc**rement rdi to advance to next argv[1] character
inc    rsi                     ; **inc**rement rsi to advance to next password character
jmp    loop                    ; jump back to the top – repeat!
```

En este caso, la instrucción de nuestro interés es la de `jmp`, puesto que este siempre va a saltar de forma incondicional.

Y el bucle terminará cuando se cumpla alguna de las condiciones.

En este problema, nos piden:

1. Detectar en que parte se encuentra el bucle con `objdump`.
2. Detectar donde se esta almacenando la contraseña.
3. Traducir la contraseña escrita en formato hexadecimal, a código ASCII cada caracter.

🚩 Flag Capturada

✓ CAPTURADA

pwn.college{0Ae7WQpSJ7pA143bEe4QwhsamaT.0V00czMywSMycjN4EzW}

1.5.10. Writing Loops

Todo bucle necesita 3 elementos:

1. Donde se encuentra actualmente.
2. Cuánto se ha avanzando.
- 3.Cuál es la condición de salida.

</>

```
.intel_syntax noprefix
mov rsi, [rsp + 16]
xor rdi, rdi
loop:
mov al, BYTE PTR [rsi]
cmp al, 0x0
je success
inc rsi
inc rdi
jmp loop

success:
mov rax, 60
syscall
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{kGPHs9T6Bo2idpKHvjEQsY42zj9.0lNyQTNywSMycjN4EzW}
```

1.5.11. Writing From a Shared Library

Lo que hemos realizado hasta ahorita son códigos que inician en `_start`, se ejecutan desde ahí y finalizan con una llamada al sistema.

Una biblioteca compartida, será un fragmento de código compilado que otro programa carga en tiempo de ejecución y al que recurre. Siendo las bibliotecas una interfaz más cómoda para interactuar.

Para poder acceder al código de la librería, vamos a utilizar la instrucción `call` `[target]`, esta instrucción empuja a la pila la dirección de la siguiente instrucción tras la instrucción `call`, para que sirva de dirección de retorno y saltamos a `[target]`.

</>

```
.intel_syntax nonprefix
.global solve
solve:
[your code, ending in an exit syscall]
```

La línea `.global solve`, va a exponer nuestro código para que otros programas puedan utilizar dicha función.

Ahora para poder compilar nuestra biblioteca compartida, se va a usar la bandera `-shared`, con el comando `ld`, de la siguiente forma:

</>

```
as -o your-solve.o your-solve.s
ld -shared -o your-solve.so your-solve.o
```

Ahora para poder pasar los parámetros a la función, se tiene que seguir las convenciones de llamada (**calling convention**).

Registro	Función al incorporarse
rdi	Primer argumento
rsi	Segundo argumento
rdx	Tercer argumento
rcx	Cuarto argumento
r8	Quinto argumento
r9	Sexto argumento
stack	Empuja los demás valores a la pila.

Tabla 1.2: Calling conventions en asm x86

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
mov rdx, rsi
mov rsi, rdi
mov rdi, 1
mov rax, 1
syscall

mov rdi, 0
mov rax, 60
syscall
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{czNSI3Wb6ITRsHns0b4e1yvvyfC4.0FN1kDNywSMycjN4EzW}
```

1.5.12. Returning a Value

En la sección anterior [1.5.11](#), terminamos la función llamando al sistema de `exit`.

Al realizar esto de aquí, termina nuestro programa, pero no regresa el control a quien hizo la llamada, para que se pueda realizar eso de ahí, se hace uso de la llamada al sistema `ret`.

En la convención de llamada de Linux x86-64, el valor de retorno se almacena en `rax`.

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
mov rax, rdi
ret
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{YN5Hs06b9fLKQ2xn10Q3Xuhap7L.0VN1kDNywSMycjN4EzW}
```

1.5.13. Calling Through a Pointer

Para este caso, se esta pasando la ubicación de la función a llamar, en el registro `rdi`, y dicha ubicación es un valor de 64 bits que nos indica la ubicación de un fragmento de código en la memoria.

</>

```
.intel_syntax noprefix
.global solve
solve:
call rdi
ret
```

Flag Capturada

✓ CAPTURADA

pwn.college{s0lh7rNWQ3zTmxtLr38hoQqTUtR.01N1kDNywSMycjN4EzW}

1.5.14. Calling Through a Pointer with an Argument

En este desafío, el valor de `rdi`, contiene el valor de la función, y a la vez si queremos pasar un valor a la función, este va a seguir el las convenciones de llamada.

</>

```
.intel_syntax noprefix
.global solve
solve:
mov rax, rdi
mov rdi, 1337
call rax
ret
```

Flag Capturada

✓ CAPTURADA

pwn.college{MLa5ffWiEMfwwHnZtH81on1VqGd.01N1kDNywSMycjN4EzW}

1.5.15. Saving Caller - Saved Registers

En la arquitectura x86-64, todos los programas comparten los mismo 16 registros por lo que la convención a llamada, establecen reglas estrictas sobre quién es responsable de preservar los datos al invocar la función.

- **Caller:** Los registros `rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`, `r9`, `r10`, `r11` se van a poder perder cuando se haga el llamado a la función, por eso mismo se tienen que guardar primero y restaurarlo después.
- **Callee:** Las funciones que se llaman no deben modificar los registros guardados por la función llamada. Por el contrario, las funciones llamadas pueden modificarlos, pero deben restablecerlos a su estado original. En x86-64, estos son `rbx`, `rbp`, `r12`, `r13`, `r14` y `r15`.

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
push rax
push rcx
push rdx
push r8
push r9
push r10
push r11
push rsi
push rdi

call rdi

pop rdi
pop rsi
pop r11
pop r10
pop r9
pop r8
pop rdx
pop rcx
pop rax

call rsi
ret
```

Flag Capturada **CAPTURADA**

```
pwn.college{kJ2f8vc_wNmA_iDpjBMeWlfv8Cr.0FM4kDNywSMycjN4EzW}
```

1.5.16. Saving Callee - Saved Registers

Ya has practicado con los registros guardados por el llamante y ahora veremos los guardados por el llamado. Los registros guardados por el llamado (rbx, rbp, r12, r13, r14, r15) deben recuperarse sin alteraciones cuando el llamado regrese. Cuando una función utiliza uno de ellos, se lo toma prestado de quien la ha llamado y debe devolverlo exactamente en el mismo estado en que lo encontró. Esto es lo que permite al llamante mantener valores de larga duración en rbx, rbp y r12-r15 a lo largo de una llamada y confiar en que seguirán ahí después.

La regla es: cuando tu función quiera utilizar estos registros, guárdalos (push) al entrar en tu función y restáuralos (pop) antes de llamar a ret.

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
push rbx
push rbp
push r12
push r13
push r14
push r15

mov rbx, 0x1337
mov rbp, 0x1337
mov r12, 0x1337
mov r13, 0x1337
mov r14, 0x1337
mov r15, 0x1337
call rdi

pop r15
pop r14
pop r13
pop r12
pop rbp
pop rbx
ret
```

Flag Capturada

✓ CAPTURADA

pwn.college{U1HPPFUtUyQeov-6iT9TeCx0MLT.0VM4kDNywSMycjN4EzW}

1.6. Endian Escapades

1.6.1. Texto - Litte-Endian

Cualquier valor que ocupe más de un byte, se divide en bytes individuales para poder almacenarse en memoria, ya que la memoria se direcciona byte a byte.

La mayoría de las arquitecturas almacenan el dígito *menos significativo* a la izquierda y el *más significativo* a la derecha. Un ejemplo sería el `0x1234`, se almacenaría como `34 12`. A este tipo de arquitecturas se les denomina Litte-Endian. Esto de aquí, se realiza por las siguiente manera:

1. Las mayoría de las operaciones aritméticas, se realiza partiendo del *litte_endian* y esto que los componentes electrónicos sea más natural.
2. La dirección de un valor es la dirección de su byte menor, por lo que leer 1 byte o 2 bytes, equivale a la misma dirección, simplemente se lee menos bytes.

1.6.2. Texto - Memory Order and Register Values

El punto en el que es más facil confundirse es la forma entre el orden de la memoria y el valor que se imprime desde un registro.

Esto no importa en sí, puesto que **x86** lo organiza y maneja por nosotros.

size of value	decimal value	hex value	big endian bytes (NOT x86)	little endian bytes (x86)
8 (1 byte)	65	0x41	41	41
16 (2 bytes)	4660	0x1234	12 34	34 12
32 (4 bytes)	1145258561	0x44434241	44 43 42 41	41 42 43 44
64 (8 bytes)	1145258561	0x0000000044434241	00 00 00 00 44 43 42 41	41 42 43 44 00 00 00 00

Tabla 1.3: Ejemplo de almacenamiento en **Little-Endian** y **Big-Endian**

memory address order:	41 42 43 44 45 46 47 48
register hex order:	48 47 46 45 44 43 42 41
rax value:	0x4847464544434241

Tabla 1.4: Ejemplo de como se almacena

1.6.3. Texto - Bytes, Words, and Friends

En x86 se va a estar trabajando siempre con diferentes tamaños de *bits*, 2, 4, 6 u 8, y es bueno conocerlo porque siempre vamos a estar trabajando con ellos:

Name	Bits	Bytes	Partial <i>rax</i> Access	Memory Access
byte	8	1	<code>mov al, [rdi]</code>	<code>mov BYTE PTR [rdi], 0x11</code>
word	16	2	<code>mov ax, [rdi]</code>	<code>mov WORD PTR [rdi], 0x11</code>
byte	32	4	<code>mov eax, [rdi]</code>	<code>mov DWORD PTR [rdi], 0x11</code>
byte	64	8	<code>mov rax, [rdi]</code>	<code>mov QWORD PTR [rdi], 0x11</code>

Tabla 1.5: Acceso a 2, 4, 6, 8 bits.

1.6.4. Sign Extension

1.6.5. Little-Endian Bytes

1.6.6. Qword by Qword

1.6.7. Dword by Dword

1.6.8. Word by Word

1.6.9. Bytes by Bytes

1.6.10. Cracking a Struct

1.6.11. Scrambled Struct